🛠️ TECH STACK (DETAILED)

FunStack – Full Stack Mini Games Hub

🎨 1. FRONTEND (Client Side)

🧩 Core Technologies:

- **HTML5**
- **CSS3**
- **JavaScript (ES6+)**
- **React.js**

⚛️ React.js (Main Framework)

💡 Why React?

- Component-based architecture
- Fast rendering (Virtual DOM)
- Easy state management
- Perfect for interactive UI (games)

🧱 Key Concepts Used:

🔹 **Components:**

- Dashboard
- Game Pages
- Navbar
- Game Cards

🔹 **Hooks:**

- useState → manage game state
- useEffect → timers, API calls

🔹 **Routing:**

- react-router-dom
  👉 Navigate between:
- Dashboard → Games

🎨 **Styling:**

Options:

- Plain CSS (recommended ✅)
- OR Tailwind CSS (optional)

💡 **UI Techniques:**

- Glassmorphism
- Neon gradients
- Animations (hover, transitions)

✨ **Animations & 3D Effects:**

🔹 **Framer Motion**

Used for:

- Card hover animations
- Button interactions
- Smooth transitions

🔹 **CSS 3D Transforms:**

- rotateX, rotateY
- perspective()
- Used in:
    - Card tilt
    - Flip animations

🌐 **API Communication:**

🔹 **Axios**

Used for:

- Sending requests to backend
- Fetching game data

⚙️ **2. BACKEND (Server Side)**

🧩 **Core Technologies:**

- **Java**
- **Spring Boot**

☕ **Spring Boot (Main Backend Framework)**

💡 **Why Spring Boot?**

- Easy setup
- REST API support
- Integration with database
- Industry standard

🧱 **Backend Architecture:**

🔹 **Controller Layer:**

Handles API requests
Example:

- /login
- /save-result

🔹 **Service Layer:**

- Business logic
- Game result processing

🔹 **Repository Layer:**

- Database interaction
- Uses JPA

🔗 **REST APIs:**

Used for:

- Authentication
- Saving game results
- Fetching history

🗄️ **3. DATABASE**

✅ **Recommended:**

- **MySQL**

💡 **Why MySQL?**

- Easy with Spring Boot
- Structured data
- Lightweight

🧱 **Tables:**

👤 **Users:**

- id
- username
- password

🎮 **Game Results:**

- id
- userId
- gameType
- score
- timestamp

🎡 **Picker Data:**

- id
- userId
- inputList
- selectedItem

🔗 **ORM:**

🔹 **Spring Data JPA**

Used for:

- Mapping Java objects → database tables
- Reducing SQL queries

🔐 4. AUTHENTICATION

🔹 **Basic Authentication (Simple):**

- Username + Password
- Stored in database

🔹 **Optional:**

- JWT (if you want advanced)

🌙 5. STATE & DATA FLOW

🔄 **Flow:**

1. User interacts with React UI
2. React sends API request (Axios)
3. Spring Boot processes request
4. Data stored in MySQL
5. Response sent back to UI

🎮 6. GAME LOGIC DISTRIBUTION

✅ **Frontend Handles:**

- Game logic (guessing, dice, quiz)
- Animations
- UI interactions

✅ **Backend Handles:**

- User authentication
- Saving results
- Fetching history

🧰 **7. DEVELOPMENT TOOLS**

👷 **Frontend:**

- VS Code
- Chrome DevTools

⚙️ **Backend:**

- IntelliJ IDEA / Eclipse

🗄️ **Database:**

- MySQL Workbench

🔗 **Version Control:**

- Git + GitHub

🚀 **8. DEPLOYMENT (OPTIONAL)**

🌐 **Frontend:**

- Netlify / Vercel

⚙️ **Backend:**

- Render / Railway

🗄️ **Database:**

- MySQL Cloud / Localhost

🧠 **FINAL TECH STACK SUMMARY**

Layer	Technology
Frontend	React.js, HTML, CSS, JS
Animation	Framer Motion, CSS 3D
Backend	Java, Spring Boot
Database	MySQL
API	REST APIs
Tools	VS Code, IntelliJ, Git